@swiftgoma/auth: usage guide

How to use the shared authentication client in a SwiftGoma app. For what the package is, what it leaves out, and the decisions behind it, read `docs/auth-package-scope.md`.

Last updated: 2026-10-27. Branch: `feat/auth-package`.

Contents

1. What you get
2. Install
3. Quick start: Expo (mobile)
4. Quick start: Next.js and Vite (web)
5. createAuth options
6. Session
7. Sign-up
8. Login
9. Two-step verification
10. Google and Apple
11. Logout and current user
12. Calling your own API routes
13. Errors
14. Server-side rendering
15. Server requirements
16. Troubleshooting
17. Known limits

---

1. What you get

The package owns **transport, session and device**. Your app owns screens and business logic.

- Sign-up by email or phone, with code confirmation and resend.
- Login by email code, phone code, password, Google and Apple.
- Two-step verification, and first-login enrollment for Support and Admin accounts.
- A session that renews itself about 2 minutes before the access token expires, plus a safety net on a 401, so users do not see expired-token errors.
- A device id, platform and name sent on every sign-up and login.
- Logout (this device or all devices) and the current user.
- Typed errors built from the server's answer. Messages are in French and come from the server unchanged.

The same app code works on mobile and web. Only the adapter changes.

Platform	Adapter	Tokens
Expo (Android, iOS)	`createNativeAdapter` from `@swiftgoma/auth/native`	Access and refresh token in secure storage
Next.js, Vite	`createWebAdapter` from `@swiftgoma/auth/web`	Cookies only; JavaScript never sees a token

2. Install

The package lives in `packages/auth` and is delivered through npm workspaces (no registry).

1. In the root `package.json`, add `"apps/*"` to `workspaces` (it holds only `"packages/*"` today). `server/` stays outside the workspace.
2. In the app's `package.json`, add the package and its peer dependency:

```
{
  "dependencies": {
    "@swiftgoma/auth": "*",
    "axios": "^1.7.0"
  }
}
```

3. Run `npm install` at the repo root.
4. Build the package before the app uses it, and again after you change it:

```
npm run build --workspace packages/auth
```

Entry points:

Import	Contains
`@swiftgoma/auth`	`createAuth`, errors, constants, types
`@swiftgoma/auth/native`	`createNativeAdapter`
`@swiftgoma/auth/web`	`createWebAdapter`, `forwardAuthHeaders`

Expo SDK 58 configures Metro for workspaces automatically and honours `package.json` `exports`. Next.js needs the package built (or `transpilePackages: ["@swiftgoma/auth"]`).

3. Quick start: Expo (mobile)

Add the Expo modules the adapter needs to the app: `expo-secure-store`, `expo-crypto` and `expo-device`.

Create one auth object for the whole app, for example in `lib/auth.ts`:

```
import * as Crypto from "expo-crypto";
import * as Device from "expo-device";
import * as SecureStore from "expo-secure-store";
import { AppState, Platform } from "react-native";
import { createAuth } from "@swiftgoma/auth";
import { createNativeAdapter } from "@swiftgoma/auth/native";

export const auth = createAuth({
  baseUrl: "https://api.swiftgoma.com",
  adapter: createNativeAdapter({
    secureStore: SecureStore,
    appState: AppState,
    os: Platform.OS,
    randomUUID: Crypto.randomUUID,
    deviceName: () => Device.modelName ?? Device.deviceName,
    userAgent: `SwiftGoma-Buyer/1.0.0 (${Platform.OS})`,
  }),
});
```

Load the saved session once when the app starts and wait for it before showing the first screen:

```
try {
  await auth.session.start();
} catch {
  // Secure storage could not be read (for example the keychain is locked). This is not a logout:
  // try again later instead of sending the user to the sign-in screen.
}
```

Then log in, and read the state to decide what to show:

```
await auth.login.emailCode.request({ email });
const result = await auth.login.emailCode.verify({ email, code });

if (result.status === "authenticated") {
  // signed in
}
```

Native adapter options

Option	Required	Meaning
`secureStore`	yes	`import * as SecureStore from "expo-secure-store"`
`appState`	yes	React Native's `AppState`
`os`	yes	`Platform.OS`; only `"ios"` and `"android"` are accepted
`randomUUID`	yes	`Crypto.randomUUID` from `expo-crypto`
`deviceName`	no	Text, or a function returning text. Shown in the user's device list, cut to 100 characters. `Device.modelName` is more useful than `Device.deviceName`, which is a generic "iPhone" on iOS 16 and later
`userAgent`	no	A stable User-Agent, printable ASCII only
`keyPrefix`	no	Prefix of the storage keys. Default `swiftgoma.auth`
`keychainService`	no	Keychain service name on iOS
`logger`	no	Receives warnings

Good to know:

- The session is stored with the `AFTER_FIRST_UNLOCK` option, so the app can still read it when it starts in the background after the phone was unlocked once.
- On iOS the keychain can survive a reinstall, so the device id may stay the same. On Android it changes after a reinstall.
- A read error from secure storage reaches your code from `session.start()`. It is never treated as a logout.
- A saved session that cannot be read is deleted and treated as signed out.

4. Quick start: Next.js and Vite (web)

Create one auth object, for example in `lib/auth.ts`:

```
import { createAuth } from "@swiftgoma/auth";
import { createWebAdapter } from "@swiftgoma/auth/web";

export const auth = createAuth({
  baseUrl: "https://api.swiftgoma.com",
  adapter: createWebAdapter({ deviceName: "Navigateur web" }),
});
```

Start it once per tab, in the browser only (in a layout `useEffect`, or at the top of your Vite entry file):

```
useEffect(() => {
  void auth.session.start();
}, []);
```

Importing `lib/auth.ts` on the server is safe: the adapter touches the browser only when it is used.

Web adapter options

Option	Meaning
`deviceName`	Text, or a function returning text. Nothing is sent by default
`keyPrefix`	Prefix of the storage keys. Starts with a letter or digit
`logger`	Receives warnings
`environment`	Replaces the browser objects (storage, locks, window, document). Used by tests

What the web adapter does for you:

- **Cookies only.** Tokens live in HttpOnly cookies. The browser's `localStorage` holds only the access token's expiry date and a device id.
- **Several tabs.** A login or logout in one tab is picked up by the others. Only one tab renews the session at a time (Web Locks), and the others adopt the result without calling the server. This matters because the server treats an old refresh token replayed after 60 seconds as theft.
- **Blocked storage.** If the browser blocks `localStorage`, the session lives in memory only and a reload signs the user out. A warning goes to the logger.
- **No User-Agent.** Browsers do not allow setting it.

5. createAuth options

```
createAuth({ baseUrl, adapter, socials, refreshMarginSeconds, requestTimeoutMs, logger });
```

Option	Default	Meaning
`baseUrl`	required	The API **origin only**, for example `https://api.swiftgoma.com`. The package adds `/api/v1`. Plain `http://` is accepted only for local and private addresses
`adapter`	required	The native or web adapter. The platform and transport must agree: web uses cookies, Android and iOS use bearer tokens
`socials`	all off	`{ google: true, apple: true }`. See section 10
`refreshMarginSeconds`	120	How long before expiry the session renews. A whole number from 10 to 600
`requestTimeoutMs`	15000	A whole number from 1000 to 120000
`logger`	silent	An object with `debug`, `warn` and `error`

A wrong value throws a `ConfigError` right away.

What `createAuth` returns:

Property	What it is
`auth.signup`	Sign-up flows (section 7, and Google and Apple in section 10)
`auth.login`	Login flows (section 8, and Google and Apple in section 10)
`auth.twoFactor`	Second step and enrollment (section 9)
`auth.logout()`, `auth.logoutAll()`, `auth.me()`	Section 11
`auth.session`	State, subscription, refresh (section 6)
`auth.socials.enabled`	The providers this app switched on
`auth.api`	The configured Axios client for your own calls (section 12)
`auth.destroy()`	Stops timers and listeners. Useful in tests and hot reload

6. Session

`auth.session` has:

Member	Meaning
`state`	The current state (below)
`start()`	Loads the saved session. Call it once and wait for it. Returns the state
`subscribe(listener)`	Calls the listener on every change. Returns an unsubscribe function
`refresh()`	Renews now. Normally you never call it
`getAccessToken()`	The current access token, renewed first if it is about to expire. `null` on the web

States:

State	Meaning
`{ status: "unknown" }`	`start()` has not finished
`{ status: "authenticated" }`	Signed in
`{ status: "unauthenticated", reason? }`	Signed out. `reason` tells why the session ended

Reasons: `logout`, `session_expired`, `session_revoked`, `token_reused`, `account_suspended`. A user who never signed in has no reason. Show a short message for `account_suspended` and `token_reused`; for the others, go to the sign-in screen.

React example (no hook is shipped yet):

```
import { useSyncExternalStore } from "react";
import { auth } from "./auth";

export function useSessionState() {
  return useSyncExternalStore(
    (onChange) => auth.session.subscribe(onChange),
    () => auth.session.state,
    () => auth.session.state,
  );
}
```

How renewal works:

- A timer renews the access token `refreshMarginSeconds` before it expires, with up to 15 seconds of random spread. It also renews when the app returns to the foreground.
- If a request still gets a 401 for an expired token, the package renews once and replays the request. A second 401 is shown to your code.
- Offline, timeout, 5xx and 429 answers keep the session and retry after 5, 15 and 30 seconds, then every 60 seconds.
- A refresh token that is invalid, expired, revoked, reused, or an account that is suspended ends the session.

7. Sign-up

Sign-up has two steps: start (the server sends a code), then verify. A successful verify signs the user in.

```
// Email
await auth.signup.email.start({
  name: "Aline Kabila",
  email: "aline@example.com",
  role: "BUYER", // "BUYER" | "SELLER" | "RIDER"
  acceptTerms: true,
  confirmAge: true,
});
const { user } = await auth.signup.email.verify({ email: "aline@example.com", code: "123456" });

// Phone
await auth.signup.phone.start({ name, phone: "+243810000000", role, acceptTerms: true,
confirmAge: true });
await auth.signup.phone.verify({ phone: "+243810000000", code });

// Ask for a new code
await auth.signup.email.resend({ email });
await auth.signup.phone.resend({ phone });
```

- `start` returns `{ method, target, expiresMinutes }`: where the code went (`"EMAIL"` or `"PHONE"`) and how long it stays valid.
- `acceptTerms` and `confirmAge` must be the literal `true`. Show the checkboxes in your UI.
- `resend` returns `{ message }`.

8. Login

Every login ends in one of three results:

`result.status`	Meaning	Next step
`authenticated`	Signed in. Has `user` and `isNewDevice`	Go to the app
`two_factor_required`	The account needs its second step	Section 9, `twoFactor.verify`
`two_factor_enrollment_required`	A Support or Admin account must set up two-step first	Section 9, `twoFactor.enroll`

Both two-step results carry `challengeExpiresIn` (seconds, about 5 minutes). The second-step token stays inside the package; you never handle it.

```
// Email code
await auth.login.emailCode.request({ email });
const result = await auth.login.emailCode.verify({ email, code });

// Phone code. `channel: "EMAIL"` sends the code to the account's email instead of an SMS.
await auth.login.phoneCode.request({ phone, channel: "EMAIL" });
const result2 = await auth.login.phoneCode.verify({ phone, code });

// Password
const result3 = await auth.login.password({ email, password });
```

Handle the result in one place:

```
switch (result.status) {
  case "authenticated":
    return goToApp(result.user);
  case "two_factor_required":
    return goToSecondStep();
  case "two_factor_enrollment_required":
    return goToEnrollment();
}
```

Sign-in is by email for the email code and the password, and by phone for the phone code. Forgot password and account recovery are not in this package.

9. Two-step verification

Second step after `two_factor_required`. Send either the app code or a backup code:

```
await auth.twoFactor.verify({ code: "123456" });
// or
await auth.twoFactor.verify({ backupCode: "ABCD-EFGH" });
```

On success the user is signed in. A wrong code throws and keeps the pending step, so the user can retry until `challengeExpiresIn` runs out.

Enrollment after `two_factor_enrollment_required` (first login of a Support or Admin account):

```
const setup = await auth.twoFactor.enroll.start();
// setup.secret       the text secret
// setup.otpauthUri   the address to show as a QR code in the authenticator app

const done = await auth.twoFactor.enroll.confirm({ code: "123456" });
// done.backupCodes   shown ONCE: show them to the user now
```

- `enroll.confirm` signs the user in and returns `{ user, isNewDevice, backupCodes }`.
- The package never saves or logs the backup codes. If your app loses them before showing them, they are gone.
- Calling the wrong step (for example `verify` during enrollment, or either call with nothing pending) throws without sending a request. The error code tells which case it is (`TWO_FACTOR_SETUP_REQUIRED`, `TWO_FACTOR_ALREADY_ENABLED`, `TWO_FACTOR_CHALLENGE_EXPIRED`).

10. Google and Apple

Social sign-in is **off by default**. Switch it on per app:

```
createAuth({ /* ... */, socials: { google: true, apple: true } });
```

- `auth.socials.enabled` lists what your sign-in screen may show, for example `["google"]`. The package draws no buttons.
- A disabled provider throws `SocialLoginDisabledError` (code `SOCIAL_LOGIN_DISABLED`) before any request is sent.
- The Google and Apple libraries stay in your app, because they need the app's own client IDs.

You give the package a function that returns the provider's ID token. The package asks the server for a one-time nonce, calls your function with it, then sends the token and the nonce:

```
const getIdToken = async (nonce: string): Promise<string> => {
  // Call your Google or Apple library here and pass `nonce` exactly as given.
  // Do not hash it: the server compares it as is.
  return idTokenFromTheProvider;
};

// Login
await auth.login.google({ getIdToken });
await auth.login.apple({ getIdToken });

// Sign-up
await auth.signup.google({ getIdToken, role: "BUYER", acceptTerms: true, confirmAge: true });
await auth.signup.apple({
  getIdToken,
  role: "BUYER",
  acceptTerms: true,
  confirmAge: true,
  name: "Aline Kabila", // optional: Apple gives the name only the first time
});
```

All four return the same login result as section 8.

- If the user cancels in the provider's screen, the library's own error reaches your code unchanged. No login request is sent.
- A function that returns an empty token throws a `TypeError`.
- The nonce is single-use and valid for 10 minutes.

Things to know:

- **App Store guideline 4.8.** If an iOS app offers Google (or another third-party login) as the main way to sign in, Apple asks for an equivalent option that limits data to name and email, lets users hide their email, and does not track for advertising. Sign in with Apple is the usual choice. The package logs a warning when an iOS build turns on Google without Apple. The guideline has exceptions (for example apps that only use their own sign-in); read the current wording before you decide.
- **Apple private relay.** A user who signed up with a real email and later uses Apple's hidden email gets a separate account. Accounts only link on a matching verified email. Explain this on your sign-in screen.
- **Apple on Android and web** needs Apple's web flow (a Services ID and a redirect page). Some apps may leave Apple off there.

11. Logout and current user

```
const { serverRevoked } = await auth.logout();
const { revoked } = await auth.logoutAll();
const me = await auth.me();
```

- `logout()` **always** signs out on this device and never throws. `serverRevoked` is `false` when the server could not be reached; you may want to tell the user.
- `logoutAll()` signs out every device and returns how many sessions ended. If the server call fails, it **throws and keeps the session** so the user can try again.
- `me()` returns the current user: id, name, email, phone, role, status, `emailVerified`, `phoneVerified`, `hasPassword`, `twoFactorEnabled`, `verificationStatus`, `createdAt`, `lastLoginAt` and `sessionId`.
- `me()` and `logoutAll()` throw `NotAuthenticatedError` when nobody is signed in.

12. Calling your own API routes

`auth.api` is the Axios client the package uses. Use it for the app's own calls so you get the same behaviour:

```
const response = await auth.api.get("/shops/mine"); //
https://api.swiftgoma.com/api/v1/shops/mine
```

- Paths are relative to `<origin>/api/v1`.
- On mobile the `Authorization: Bearer` header is added for you. On the web, cookies are sent (`withCredentials`) and no header is added.
- An expired token is renewed and the request is replayed once.
- Every failure becomes a typed error (section 13). A cancelled request passes through unchanged.
- Add `skipAuth: true` to a request's config for a call that must not carry a token.

13. Errors

Every failure is an `ApiError` or a subclass. Read `error.code` to decide what to do, and show `error.message` (French, from the server) to the user.

Class	When
`ApiError`	Base class. Has `code`, `status`, `message`, `details`, `requestId` and the original Axios error as `cause`
`UnauthorizedError`	401
`ForbiddenError`	403
`NotFoundError`	404
`ConflictError`	409
`ValidationError`	422. `error.fields` is a list of `{ field, message }`
`TooManyRequestsError`	429. `error.retryAfterSeconds` may be set
`ServerError`	5xx
`NetworkError`	The server could not be reached
`TimeoutError`	The server took too long
`NotAuthenticatedError`	You asked for something that needs a session (`NOT_AUTHENTICATED`)
`SocialLoginDisabledError`	A provider the app did not switch on

```
import { hasErrorCode, isApiError } from "@swiftgoma/auth";

try {
  await auth.login.password({ email, password });
} catch (error) {
  if (hasErrorCode(error, "INVALID_CREDENTIALS")) return showWrongPassword();
  if (isApiError(error)) return showMessage(error.message);
  throw error;
}
```

Codes you will meet most often: `INVALID_CREDENTIALS`, `OTP_INVALID`, `OTP_COOLDOWN`, `OTP_HOURLY_LIMIT`, `OTP_ATTEMPTS_EXCEEDED`, `ACCOUNT_EXISTS`, `ACCOUNT_NOT_FOUND`, `ACCOUNT_SUSPENDED`, `ACCOUNT_PENDING_APPROVAL`, `TWO_FACTOR_INVALID`, `TWO_FACTOR_LOCKED`, `TWO_FACTOR_CHALLENGE_EXPIRED`, `GOOGLE_TOKEN_INVALID`, `APPLE_TOKEN_INVALID`, `SESSION_REVOKED`, `NETWORK_ERROR`, `TIMEOUT`. The full list is `ERROR_CODES` in the package.

14. Server-side rendering

For a Next.js page that must read the signed-in user on the server, forward the visitor's session cookies to the API:

```
import { headers } from "next/headers";
import { forwardAuthHeaders } from "@swiftgoma/auth/web";
import { auth } from "@/lib/auth";

const cookie = (await headers()).get("cookie");
const response = await auth.api.get("/auth/me", {
  headers: forwardAuthHeaders({ cookie, origin: "https://app.swiftgoma.com" }),
});
```

- Only the two session cookies (`__Host-swg_access` and `swg_refresh`) are forwarded; every other cookie is dropped. With no session cookie, no header is added.
- `origin` is your web app's own origin. The API needs it on cookie-authenticated writes.
- **Host rule.** The access cookie is `__Host-` prefixed, so it belongs to one host only. This works only when the browser and your Next.js server reach the API under the same host, for example through a rewrite or reverse proxy. If the API is on another host, call it from the browser.
- A refresh done on the server cannot set the visitor's cookies. Keep renewals in the browser and use this helper for reads.

15. Server requirements

The package assumes the SwiftGoma API (`server/`) is set up like this:

- Each web app origin is listed in `CORS_ORIGINS` .
- The web app and the API share a registrable domain (cookies are `SameSite=Lax`), or the web app calls the API through its own server.
- Google and Apple are configured on the server (`GOOGLE_CLIENT_IDS` , `APPLE_CLIENT_IDS`) before you switch them on in an app.
- Pending server change: login, verify and enrollment answers should return `expiresIn` . Until then the web assumes a 15-minute access token (see section 17).

16. Troubleshooting

Symptom	Likely cause and fix
`ConfigError: baseUrl must be the API origin only`	Remove the path from `baseUrl`; the package adds `/api/v1`
`ConfigError: baseUrl must use https`	Plain `http://` works only for localhost and private addresses
`ConfigError` about transport	Web must use the web adapter, Android and iOS the native adapter
Every web refresh fails and retries each minute	The app's origin is missing from the server's `CORS_ORIGINS` (`CORS_NOT_ALLOWED`)
Web users are signed out on every reload	The browser blocks `localStorage`; look for the adapter's warning in the logger
`session.start()` throws on mobile	Secure storage could not be read. Retry later; do not treat it as a logout
Social button does nothing	The provider is off: check `auth.socials.enabled`
`SESSION_REVOKED` after using another device	The user signed out everywhere, or a stolen token was detected
Next.js cannot import the package	Build it first, or set `transpilePackages: ["@swiftgoma/auth"]`

17. Known limits

- **Web early refresh.** The server's web login answers carry no `expiresIn` yet, so the web assumes 15 minutes and finds out about an earlier expiry through the 401 safety net. Mobile reads the token's own expiry and renews on time.
- **User-Agent on a real device** has not been verified yet. It needs the first Expo app.
- **Contract tests** that run the package against the real server exist but have not been run yet. The unit tests use fake servers.
- **Not in the package:** passkeys, forgot password, account recovery, linking Google or Apple to an existing account, profile and security screens, push token registration, any UI, translations. React hooks may come later.
- tsdown is pre-1.0 and TypeScript is pinned to `~6.0.3`; expect small build option changes.